

Clustering & Unsupervised Learning

From Safaricom customer segments to Kenyan agricultural zones, discover hidden patterns without labels.

By end of this week, you will be able to

- Explain the difference between supervised and unsupervised learning
- Describe how K-Means clustering assigns data points to groups
- Apply the elbow method and silhouette score to choose the right K
- Build and interpret a hierarchical clustering dendrogram
- Visualise clusters using scatter plots and PCA-reduced dimensions
- Compare K-Means and hierarchical clustering and choose between them
- Apply clustering to a real-world East African dataset

1. The Shift to Unsupervised Learning

Safaricom has 40 million subscribers. Nobody sat down and manually labelled each one as "heavy data user", "SMS-only user", or "voice-heavy rural user." Safaricom's analytics team let an algorithm find those groups on its own — by looking at usage patterns and letting structure emerge from the data. That is unsupervised learning.

Aspect	Supervised (Weeks 3-4)	Unsupervised (This week)
Target column (y)	Required	None
What it learns	Mapping from X to y	Structure within X alone
Output	Prediction or class label	Group membership or pattern

Example	Loan default Yes/No	Customer segments
Evaluation	Accuracy, RMSE, F1	Silhouette score, inertia

Three main use cases for clustering in Kenya/Africa:

- Customer segmentation: Safaricom, Equity Bank, Jumia Kenya targeting
- Health facility grouping: MOH clustering dispensaries by patient load, equipment, and service type for resource allocation
- Agricultural zoning: grouping Kenyan counties by rainfall, soil type, and crop yield for targeted subsidy policy

2. What is K-Means Clustering?

The algorithm in plain English:

1. Choose K (the number of clusters you want)
2. Randomly place K centroids in the feature space
3. Assign each data point to its nearest centroid (Euclidean distance)
4. Move each centroid to the mean position of all points assigned to it
5. Repeat steps 3–4 until assignments stop changing (convergence)

Kenyan market vendor analogy: imagine K market stalls set up randomly in Nairobi. Every customer walks to the nearest stall. The stall owner then moves to the average position of their customers. Customers re-walk to the nearest stall. Repeat until no one moves. The final positions are the cluster centres.

Lab 1 – K-Means on M-Pesa Agent Data • Run in Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Simulated M-Pesa agent usage data
np.random.seed(42)
n = 300
data = pd.DataFrame({
    "monthly_transactions": np.concatenate([
        np.random.normal(150, 30, 100), # low-volume agents
        np.random.normal(500, 60, 100), # mid-volume agents
        np.random.normal(950, 80, 100) # high-volume agents
    ]),
    "avg_transaction ksh": np.concatenate([
        np.random.normal(800, 200, 100),
        np.random.normal(2500, 400, 100),
        np.random.normal(6000, 800, 100)
    ])
})

# Scale features (critical for distance-based algorithms)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(data)

# Fit K-Means with K=3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)
data["cluster"] = kmeans.labels_
```

```
# Visualise
colors = ["#1D9E75", "#EF9F27", "#D85A30"]
fig, ax = plt.subplots(figsize=(8, 5))
for i, grp in data.groupby("cluster"):
    ax.scatter(grp["monthly_transactions"], grp["avg_transaction_ksh"],
               label=f"Cluster {i}", color=colors[i], alpha=0.6)
ax.set_xlabel("Monthly Transactions")
ax.set_ylabel("Avg Transaction (Ksh)")
ax.set_title("M-Pesa Agent Clusters (K=3)")
ax.legend()
plt.tight_layout()
plt.show()

# Cluster summaries

print(data.groupby("cluster").mean().round(0))
```

Always scale your features before K-Means. The algorithm uses Euclidean distance, so a feature in Ksh millions will completely dominate one in single digits. StandardScaler brings all features to the same scale.

3. Choosing K — The Elbow Method

The problem: K-Means requires you to specify K before running. Too few clusters miss real sub-groups. Too many create meaningless micro-segments.

Introduce inertia: the sum of squared distances from each point to its nearest centroid. As K increases, inertia always decreases — at K=N (one cluster per point) inertia is zero. The elbow is the point where adding more clusters stops providing meaningful gains.

Lab 2 – Elbow Method [Run in Colab](#)

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertias = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.figure(figsize=(9, 4))
plt.plot(K_range, inertias, marker="o", color="#1D9E75", linewidth=2)
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia (Within-cluster SSQ)")
plt.title("Elbow Method – M-Pesa Agent Clustering")
plt.xticks(K_range)
plt.tight_layout()

plt.show()
```

The elbow is not always obvious. When the curve is smooth with no clear bend, combine it with the silhouette score. The two methods together give a much stronger basis for your choice of K.

4. Evaluating Clusters – Silhouette Score

Explain the silhouette score conceptually:

For each data point, it measures:

- How similar it is to others in its own cluster (cohesion)
- How different it is from points in the nearest other cluster (separation)

Score ranges from -1 to +1:

- +1: point is well-matched to its cluster, far from others
- 0: point sits on the boundary between two clusters
- -1: point is likely assigned to the wrong cluster

Lab 3 – Silhouette Score • [Run in Colab](#)

```
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans

sil_scores = []
for k in range(2, 11): # silhouette requires at least 2 clusters
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    sil_scores.append(silhouette_score(X_scaled, labels))

plt.figure(figsize=(9, 4))
plt.plot(range(2, 11), sil_scores, marker="s",
         color="#EF9F27", linewidth=2)
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs K")
plt.tight_layout()
plt.show()

best_k = range(2, 11)[sil_scores.index(max(sil_scores))]

print(f"Best K by silhouette: {best_k} (score: {max(sil_scores):.3f})")
```

Compare the elbow plot and the silhouette plot side by side. If both point to $K=3$, you have strong evidence. If they disagree, examine both K values: visualise the clusters and ask which segmentation makes more business sense for your use case.

5. Visualising Clusters with PCA

The problem: when you have more than 2 features, you cannot directly scatter-plot the clusters. PCA (Principal Component Analysis) reduces any number of features to 2 dimensions for visualisation, while preserving as much variance as possible.

Explain PCA in one sentence: it finds the directions of maximum variance in the data and projects all points onto the top two, giving you a 2D view of a high-dimensional space.

Lab 4 – PCA Visualisation • [Run in Colab](#)

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# After fitting KMeans on scaled data with best_k clusters:
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print(f"Variance explained by PC1 + PC2: "
      f"{pca.explained_variance_ratio_.sum():.1%}")

plt.figure(figsize=(8, 5))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1],
                      c=kmeans.labels,
                      cmap="RdYlGn", alpha=0.6, edgecolors="white", s=50)
plt.colorbar(scatter, label="Cluster")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)")
```

```
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)")
plt.title("Cluster Visualisation – PCA Projection")
plt.tight_layout()
```

```
plt.show()
```

PCA for visualisation is not the same as PCA for modelling. Here you are only using it to see the clusters in 2D. The actual K-Means model was trained on all features — the PCA projection just helps you spot separation visually.

6. Hierarchical Clustering

K-Means requires K in advance. Hierarchical clustering does not. It builds a tree of merges (or splits) that you can cut at any level to get any number of groups.

Two approaches:

- Agglomerative (bottom-up): start with N clusters (one per point), then repeatedly merge the two closest clusters until one remains. Most common.
- Divisive (top-down): start with one cluster and keep splitting. Rarely used.

Linkage methods (how "distance between clusters" is defined):

- Ward: minimises the increase in total within-cluster variance after merging. Best for compact, similarly-sized clusters. Default choice.
- Complete: distance = maximum distance between any two points in the clusters. Creates compact clusters, sensitive to outliers.
- Average: distance = average of all pairwise distances. A balance between Ward and complete.

Kenyan example: MOH clustering health facilities in Western Kenya. Rather than specifying "we want exactly 4 types of facility," hierarchical clustering reveals the natural groupings — the dendrogram shows where clear separations exist between facility capability levels.

Lab 5 – Hierarchical Clustering & Dendrogram [Run in Colab](#)

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.preprocessing import StandardScaler

# Use a smaller sample for dendrogram readability
sample = X_scaled[:80]

# Build the linkage matrix using Ward method
Z = linkage(sample, method="ward")

# Plot the dendrogram
plt.figure(figsize=(14, 5))
dendrogram(Z, leaf_rotation=90, leaf_font_size=7,
           color_threshold=0.7 * max(Z[:, 2]))
plt.title("Dendrogram – Ward Linkage (Health Facility Sample)")
plt.xlabel("Sample Index")
plt.ylabel("Merge Distance")
plt.tight_layout()
plt.show()

# Cut the tree to get flat clusters (e.g. 3 groups)
labels_hier = fcluster(Z, t=3, criterion="maxclust")

print(f"Cluster sizes: {pd.Series(labels_hier).value_counts().to_dict()}")
```

Explain the dendrogram: each vertical line is a merge. The height of the line shows how dissimilar the merged clusters were. Long vertical lines (big jumps in height) indicate natural gaps in the data — that is where you should cut.

7. K-Means vs Hierarchical — Choosing

Factor	K-Means	Hierarchical
Need to specify K	Yes — upfront	No — cut dendrogram later
Scales to large data	Yes — fast even on millions	No — slow on >10,000 rows

Cluster shape	Spherical/convex only	Any shape
Reproducible	Sensitive to initialisation	Deterministic
Interpretability	Cluster centres (centroids)	Dendrogram tree
Best for	Large datasets, clear segments, customer/user segmentation	Exploratory, small data, biology, facility grouping

In practice, start with K-Means for large datasets. Use hierarchical clustering to explore structure on a representative sample — the dendrogram will suggest a sensible K that you can then use in K-Means on the full data.

8. Interpreting and Profiling Clusters

Getting a cluster label is only the start. The real value comes from profiling each cluster — understanding what makes it distinct — and translating that into a name or business action.

Lab 6 — Cluster Profiling • [Run in Colab](#)

```
import matplotlib.pyplot as plt
import pandas as pd

# After fitting K-Means on the M-Pesa agent data (Section 2):
profile = data.groupby("cluster").agg(
    count=("monthly_transactions", "size"),
    avg_transactions=("monthly_transactions", "mean"),
    avg_value_ksh=("avg transaction ksh", "mean")
).round(0)

print(profile)

# Assign business names based on profile
cluster_names = {
    0: "Low-Volume Agents",
    1: "Mid-Volume Agents",
    2: "High-Volume Agents"
}

data["segment"] = data["cluster"].map(cluster_names)
print(data["segment"].value_counts())

# Bar chart of average transaction value by segment
profile["avg_value_ksh"].plot(kind="bar",
    color=["#1D9E75", "#EF9F27", "#D85A30"],
    figsize=(7, 4))
plt.title("Average Transaction Value by Agent Segment")
```

```
plt.ylabel("Average Transaction (Ksh)")
plt.xticks(rotation=0)
plt.tight_layout()
```

```
plt.show()
```

A cluster is only useful if it leads to a decision. "Group A has higher transaction values" is an observation. "Assign a dedicated account manager to Group A agents and offer them preferential float rates" is an action. Always profile your clusters and ask: what would we do differently for each group?

9. K-Means Limitations

K-Means makes three assumptions that are worth knowing:

- Clusters are spherical (circular in 2D). It struggles with elongated, crescent-shaped, or interleaved clusters.
- Clusters are roughly equal in size. A very small dense cluster and a large spread-out one will be incorrectly merged.
- K must be set in advance. The elbow and silhouette methods help but are not always definitive.

Briefly introduce DBSCAN as an alternative for non-spherical clusters:

"DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points that are densely packed together and labels outliers as noise. It

does not require K in advance. We do not cover it in depth here, but it is worth exploring if K-Means produces poor results on your dataset."

Lab 7 – K-Means vs DBSCAN on Crescent Data • [Run in Colab](#)

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN

X_moon, _ = make_moons(n_samples=200, noise=0.08, random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

km_labels = KMeans(n_clusters=2, random_state=42,
n_init=10).fit_predict(X_moon)
axes[0].scatter(X_moon[:,0], X_moon[:,1], c=km_labels, cmap="RdYlGn", s=30)
axes[0].set_title("K-Means on Crescent Data (fails)")

db_labels = DBSCAN(eps=0.2, min_samples=5).fit_predict(X_moon)
axes[1].scatter(X_moon[:,0], X_moon[:,1], c=db_labels, cmap="RdYlGn", s=30)
axes[1].set_title("DBSCAN on Crescent Data (succeeds)")

plt.tight_layout()

plt.show()
```

Urban Air Pollution Challenge

Air quality monitoring across African cities. Ignore the prediction target. Instead, apply K-Means clustering to discover natural groupings. Profile each cluster and give it a descriptive label.

[Open Zindi Challenge](#)

Midpoint Reset

Next week is a rest-and-catch-up week. No new content, no new quiz, no mandatory submissions. Use it deliberately:

- Revisit any concept that felt unclear — clustering, preprocessing, metrics
- Explore the Zindi leaderboards: see how your clustering compares to others
- Rest. Week 6 introduces neural networks and the programme accelerates.

Come back for Week 6 fresh.

Resources

Official Documentation

KMeans

Scikit-learn

Free

Clustering User Guide

Scikit-learn

Free

Silhouette Score

Scikit-learn

Free

`scipy.cluster.hierarchy`

SciPy

Free

Interactive Courses

Cluster Analysis in Python

DataCamp

Free via Ngao

Unsupervised Learning in Python

DataCamp

Free via Ngao

Intro to Machine Learning (clustering section)

Kaggle Learn

Free

Books & Deep Dives

ISLR Chapter 12: Unsupervised Learning

Free PDF

Free

Python DS Handbook 5.11: K-Means Clustering

Free Online

Free

Hands-On ML Chapter 9: Unsupervised Learning

Free Notebook

Free

Africa Datasets for Practice

Urban Air Pollution

Zindi — This week's challenge

Africa

Financial Inclusion in Africa

Zindi — Revisit: cluster respondents by financial behaviour

Africa

Kenya Open Data Portal

Academic References

1. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, 1, 281–297. — The original K-Means paper.
2. Ward, J.H. (1963). Hierarchical grouping to optimize an objective function. Journal of the American Statistical Association, 58(301), 236–244. doi.org/10.2307/2282967 — The Ward linkage criterion.
3. Rousseeuw, P.J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65. [doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7) — The silhouette score paper.
4. Ester, M. et al. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise (DBSCAN). KDD-96, 226–231. — The DBSCAN paper referenced in Section 9.
5. James, G. et al. (2023). An Introduction to Statistical Learning. Springer. Free PDF at statlearning.com — Chapter 12 covers this week's content in depth.